

blackwell-nanogpt

An 85M-parameter language model trained end to end on consumer Blackwell hardware: pretraining, SFT, GRPO, and agentic tool-use RL — with the result that did not hold up

Alex Huo (with Claude Code)

10 September 2026

Abstract

This report documents a complete language-model training pipeline built and run on a single consumer-class NVIDIA RTX PRO Blackwell GPU: a 20.40-billion-token pretraining run of an 85M-parameter weight-shared transformer in NVFP4/BF16, followed by supervised fine-tuning, GRPO with execution-verified rewards, and multi-turn agentic reinforcement learning against a real repository with MCP-shaped tools. Every stage ran on real hardware and moved its target metric: held-out cross-entropy $2.1817 \rightarrow 2.1155$; masked SFT loss $1.8564 \rightarrow 0.9830$ with chat-format compliance $0\% \rightarrow 100\%$; agentic episode reward $0.183 \rightarrow 0.656$ on a control task set.

The report’s most important content is a *retraction*. The headline agentic result — a 70% solve rate on repository repair tasks — was re-probed against harder breakages of the same tasks and collapsed to **0 of 120 episodes**, twice. The metric had been measuring the reversal of single-token mutations, not the ability to repair code. A follow-up pass@12 probe found 0/240 samples with *any* partial credit, and that in 236 of 240 episodes the model wrote the unchanged stub back to disk. We report how the artifact was found, why the failure was invisible for several days, and why the planned next step — distilling trajectories from a stronger teacher agent — was designed and then deliberately not built.

The engineering conclusion is that at this scale an agentic *tool workflow* and the ability to *write code* are separable capabilities, and only the first is reachable.

1 What was built, and what it cost

Stage	Outcome on real hardware	Gate
Tokenizer + data	32,784-token BPE, 20.4B-token corpus	—
1. Pretraining	held-out CE $2.1817 \rightarrow 2.1155$ over 1.245M steps	yes
2. SFT	masked CE $1.8564 \rightarrow 0.9830$; format $0\% \rightarrow 100\%$	yes
3. GRPO (control set)	76 update steps, reward $\rightarrow 0.334$	yes
3. GRPO (MBPP)	52 update steps from 300 rollouts; 9/300 solved	partial
4. Agentic RL (control)	reward $0.183 \rightarrow 0.656$, solved $15.6\% \rightarrow 52.5\%$	yes
4. Agentic RL (repo)	<i>apparent</i> solve $42.5\% \rightarrow 50.9\% / 62.5\%$	retracted
5. Hardness audit	0/120 solved on honest breakage tiers	—

Table 1: Every stage ran, and every stage moved its metric. Stage 4’s repository result did not survive an audit; §6 is that audit.

Hardware and cost. All work ran on rented AWS spot capacity: an RTX PRO 4500 Blackwell (32 GB, g7.2xlarge) for most of the pipeline and an RTX PRO 6000 (96 GB) for the throughput-critical portion of pretraining. The PRO 4500 sustained 45,146 tok/s. The PRO 6000 initially reached 77,900 tok/s ($1.726\times$), and after batch-size tuning to fill its 96 GB it sustained a **mean of 119,807 tok/s** (median 119,848, max 121,020) across the final 4.0B tokens — a $2.65\times$ speedup over the PRO 4500, cross-checked against wall clock (9.27 h for 4.0B tokens implies 119,770 tok/s, matching the instantaneous counter). Total spend was approximately \$180 in GPU time, of which roughly \$2.30 was wasted on instances left idle after a run finished; that operational miss recurred twice and is recorded because it is the sort of thing a summary usually omits.

A stale number in this project’s own README

The repository’s README claimed PRO 6000 figures were *extrapolated* by $1.726\times$ and that “nothing here has run on a PRO 6000”. Both statements were true when written and both were still there long after a PRO 6000 had run the final 4B tokens at 120k tok/s. The number was never re-read against the metrics that disproved it. This is the same failure as §6, in a much less costly place: a claim, once written down, stops being checked.

Model. 85M parameters, $d_{\text{model}} = 1536$, weight-shared depth with one transformer block applied $n_{\text{loops}} = 8$ times, 2048-token context, trained in NVFP4 via NVIDIA Transformer Engine on sm_120. The parameter count is deliberately small: the project’s stated purpose was to validate a *pipeline* that could later be pointed at a larger base model, not to produce a competitive model.

2 Pretraining

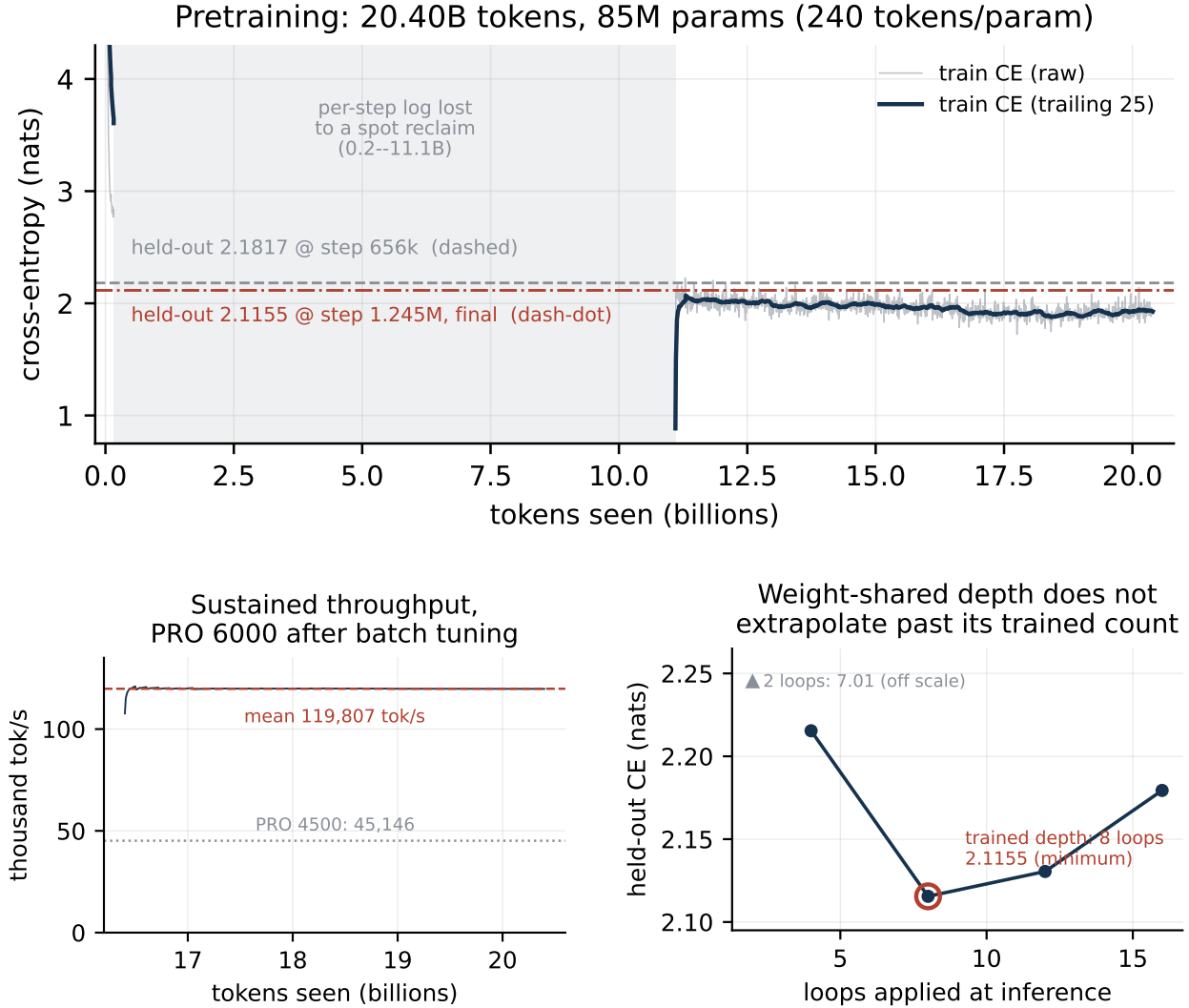


Figure 1: Top: the pretraining curve, plotted on real token counts from both metric sources. *The shaded band is a hole in the data, not a plateau:* the per-step log between 0.10B and 11.10B tokens was lost to a spot reclaim, and the line is broken rather than interpolated — drawing through it produced a convincing 11B-token plateau that never happened. The single excursion below 1.0 nat at the right edge of the band is the first logged step after that resume. The raw series is drawn underneath the trailing mean deliberately — see §6.1. **Bottom left:** sustained throughput over the final 4.0B tokens. **Bottom right:** varying the number of loop applications at inference time. Held-out loss is minimised at exactly the trained depth (8) and rises at both 4 and 16; weight sharing does not buy free depth extrapolation. The 2-loop point (7.01) is annotated rather than plotted, since clipping it silently would misrepresent the range.

The model reached 240 tokens per parameter — well past Chinchilla-optimal, and intentionally so, because the downstream stages needed the strongest possible base at a fixed small size. Three checkpoints were evaluated on a genuinely held-out split:

Checkpoint	Step	Held-out CE	Code split	Text split
baseline	656,711	2.1817	1.0598	3.5910
mid	910,397	2.1325	1.2693	3.4211
final	1,245,117	2.1155	1.1979	3.3643

Table 2: Held-out cross-entropy in nats. The code split improves from baseline to final but is *worse* at the mid checkpoint — the corpus mixture shifted mid-run, and the aggregate number hides that.

2.1 Two numerical findings that changed the whole pipeline

BF16 parameters cannot hold optimizer state

Transformer Engine’s NVFP4 path requires BF16 parameters. BF16 spacing near a value x is $2^{\lfloor \log_2 x \rfloor - 7}$, so any AdamW update smaller than that rounds away entirely. The consequence was measured, not theorised: after 657,000 steps, all five RMSNorm gain vectors were still *bit-exactly* 1.0, and at the post-training learning rates only 8% (SFT) and 2% (GRPO) of parameter elements could move at all. Without a fix, SFT was training 8% of the model and GRPO 2%. *Fix:* `blackwell_lm/optim.py` keeps FP32 master weights and copies back to BF16 after each step, at ~3% memory overhead.

The fix had a trap of its own. Applying masters to *every* parameter, including the 1-D gains frozen for 657k steps, made the model **worse**: held-out loss rose by 0.052 as stale momentum discharged into weights that had never moved. The run was rolled back — both local weights and the S3 mirror — to step 656,711, and 1-D gains were left frozen. A second lesson sits inside the first: an early error in this report’s drafting put that regression at +0.21; the difference was drift from an evaluation set contaminated with early-corpus data, and building a properly held-out set reduced it to +0.05.

Flat loss at high learning rate is not saturation

The training loss was flat for over 1B tokens and was described — wrongly — as the model having saturated. An anneal probe that dropped the learning rate to its schedule floor recovered 0.053 nats in 41M tokens. The model was learning-rate-limited, not capacity-limited. “Saturated” is a claim about capacity and requires an anneal probe to support it.

3 Supervised fine-tuning

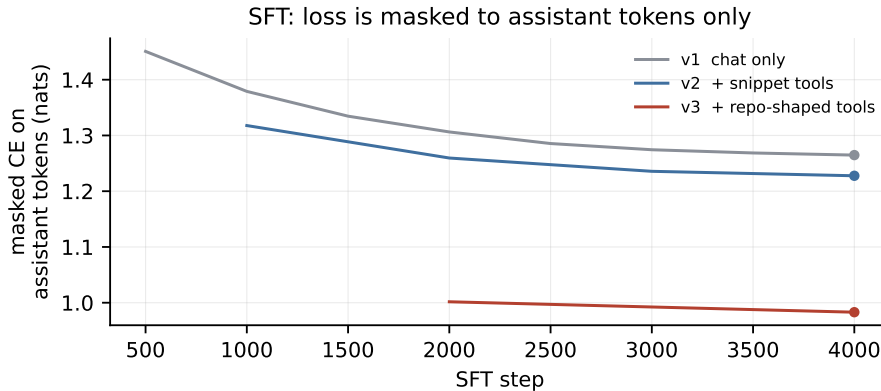


Figure 2: Masked SFT loss across three data recipes. Loss is computed on assistant tokens only, via the existing fused cross-entropy kernel’s `reduction="none"` path — no kernel changes were required.

3.1 RL cannot bootstrap a convention the policy never emits

The model emitted `write_file` calls with *no* content argument 19 times out of 19. Three interventions were tried, in order:

Run	Data	Masked CE	Format compliance	Tool-call rate
v1	chat only	2.0056 → 1.2648	50% → 100%	0% → 0%
v2	+ synthetic snippet tools	2.0128 → 1.2277	0% → 100%	0% → 50%
v3	+ repo-shaped tools	1.8564 → 0.9830	0% → 100%	0% → 100%

Table 3: Chat-format compliance went from nothing to perfect. The tool-call rate is the load-bearing column: see §3.1.

1. **A more informative error message.** The tool’s validator was changed to return, as data, the exact expected call shape: `write_file` requires `'content'`. Expected call: `{"name": ..., "args": ...}`. Result: 0/18. The model could not act on an instruction it could read.
2. **Reinforcement learning.** With no correct call ever sampled, every group had identical reward, every group was dropped as zero-variance, and the policy took 0 update steps toward the convention.
3. **Demonstrations.** Repository-shaped SFT trajectories — `read_file` → `write_file(path, content)` → `run_tests` → answer, with every tool output produced by real dispatch rather than written by hand — took the tool-call rate to 100%.

This pattern recurred three independent times in the project and is its most portable lesson: **RL sharpens a distribution, it cannot create support in it.** If the policy never emits the behaviour, reward shaping has nothing to reinforce, and the fix is demonstrations.

4 GRPO with execution-verified rewards

Rewards come from executing the generated code against hidden tests — never from a learned reward model. Sequence-level GSPO ratios, Schulman’s *k3* KL estimator, DAPO Clip-Higher, and zero-variance group dropping.

Task set	Rollout steps	Update steps	Fully solved	Final mean reward
control (12 tasks)	40	76	30	0.334
MBPP (300 tasks)	300	52	9/300	0.0082

Table 4: The control set separates a pipeline defect from a capability limit. The same code that takes 76 update steps on solvable tasks takes 52 on MBPP because 91% of MBPP groups are zero-variance — the model simply cannot solve those problems, so there is no gradient. That is a model limit, and the control run proves it is not a bug.

Two under-powered probes, both corrected

A tool-call gate measured on 4 prompts reported rates of exactly 0, 25, 50, 75 and 100% — the only values 4 samples can produce. Separately, an SFT A/B at $n=60$ showed +7pp and was dismissible as noise; re-run at $n=240$ the same comparison gave +13pp and was significant. Rate probes in this report carry standard errors for that reason.

5 Agentic reinforcement learning

Each task becomes a real directory containing a deliberately broken `solution.py`. The agent has four MCP-shaped tools — `list_dir`, `read_file`, `write_file`, `run_tests` — declared with real JSON Schema, and the system prompt is *generated from those declarations* so it cannot drift from the dispatcher.

The reward reads the file, not the transcript

An earlier design extracted code from the model’s final message. An episode that ran out of turns mid-tool-call had nothing to extract, scored a flat 0, made every group zero-variance, and produced 100 episodes and 0 updates. Scoring the *file the agent leaves behind* is both more faithful to what a coding agent is for and structurally immune to answer-extraction bugs. The asymmetry is intended: an agent that edits correctly and says nothing scores full marks, and one that describes a perfect fix without writing it scores zero.

Two further safety properties were adopted after being violated: scenarios are **verified breakable** (the reference must pass and the broken file must fail, in the sandbox, before a scenario is used — otherwise an unsolvable task and a broken policy look identical), and **the harness owns the grader**. The model was observed inventing its own tests and passing them; `run_tests(code)` now ignores model-supplied tests entirely.

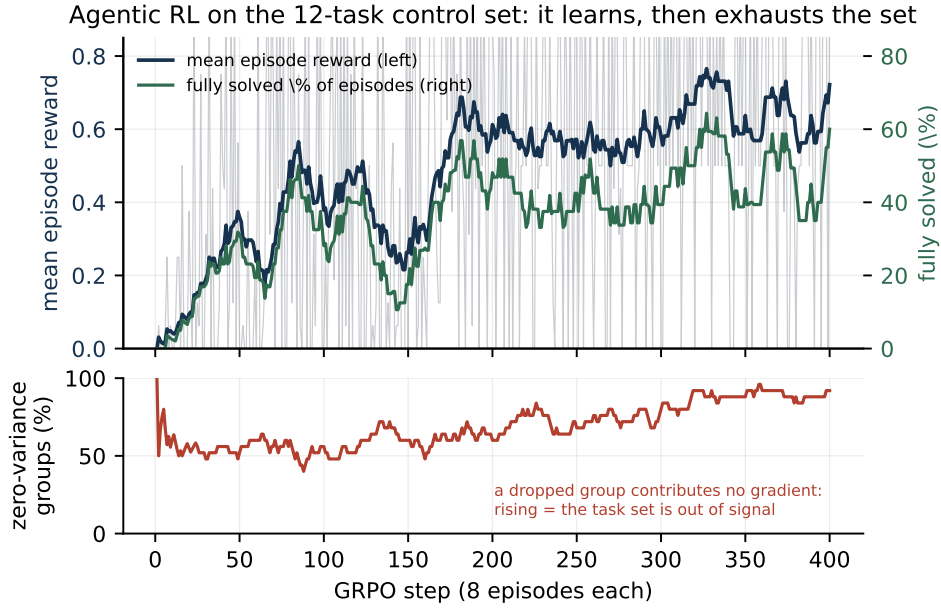


Figure 3: Agentic RL on the 12-task control set: reward 0.183 $\rightarrow$ 0.656 and fully-solved 15.6% $\rightarrow$ 52.5% over 400 steps (3,200 episodes, 3,413 update steps). The lower panel is the honest caveat — zero-variance groups rise to $\sim$ 70% as a 12-task set runs out of signal.

5.1 RL removes any behaviour the reward does not pay for

Because the reward reads only the final file, calling `run_tests` earned nothing and cost a turn. The policy responded exactly as instructed.

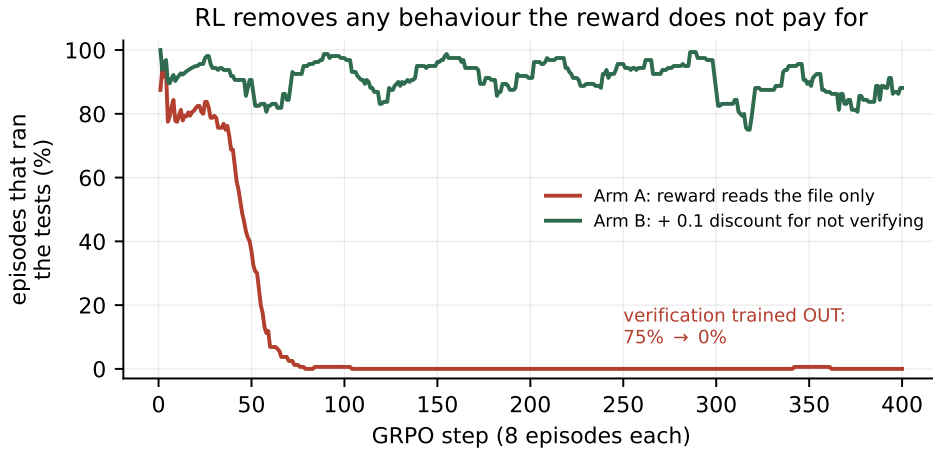


Figure 4: Arm A pays only for the file. Self-verification is trained *out*: episodes running the tests fall from 75.3% to 0.0%, and tool errors fall 433 $\rightarrow$ 58 in lockstep because the errors *were* the `run_tests` calls. Reward still rose throughout. Arm B adds a 10% discount for finishing unverified and holds verification at 86%.

The fix is an incentive, not a patch to the policy. The first implementation was an additive bonus clamped to $[0, 1]$ — a no-op, since it can only fire when the reward is already 1.0 and then clamps straight back. Worse, the unit test passed it, because the test checked the clamp rather than checking that the incentive changed any ordering. Reimplemented as a *discount* for not verifying, with a test asserting the strict ordering verified (1.00) $>$ blind (0.90) $>$ verified-broken (0.33).

	Arm A (file-only)	Arm B (+verify discount)
mean episode reward (first 10% → last 10%)	0.480 → 0.581	0.663 → 0.661
fully solved	42.5% → 50.9%	63.4% → 62.5%
wrote a file	79.4% → 80.3%	91.2% → 86.2%
ran the tests	75.3% → 0.0%	93.4% → 86.2%
own verification passed	—	63.7% → 68.4%
tool errors (total)	433	58

Table 5: 3,200 episodes per arm. Note Arm B’s reward is **flat**: 0.663 → 0.661. It began high because its SFT checkpoint was better, and 400 steps of RL added nothing — another instance of demonstrations outperforming RL at this scale.

6 The hardness audit, and the retraction

Arm B’s ~70% solve rate was the project’s apparent headline result and the intended foundation for the next stage. Before building on it, the same checkpoint was re-probed against *harder breakages of the same tasks*.

Tier	How <code>solution.py</code> is broken	n	Solved	Wrote a file
mutate	one regex substitution (+ → −)	120	59% ± 4	86% ± 3
multi	two substitutions at once	120	60% ± 4	86% ± 3
stub	function body replaced by <code>pass</code>	120	0% ± 0	98% ± 1
swap	a <i>different</i> task’s real solution	120	0% ± 0	93% ± 2

Table 6: stub and swap are the honest tiers: neither can be solved by noticing a suspicious-looking token.

The 70% was an artifact. It is withdrawn.

Zero of 120 episodes, on two independent honest tiers. The metric was measuring “*revert the token that looks odd*”, not “*repair the code*”. A pass@12 follow-up on stub (20 tasks × 12 samples = 240 episodes) found:

- pass@1 = 0.0% (0/240 samples)
- pass@12 = 0.0% (0/20 tasks *ever* solved)
- **any partial credit** = 0.0% (0/240 samples with reward > 0)
- final file contents: 236 **still the unchanged stub**, 3 syntax errors, 1 valid Python

Three things follow, and each is stronger than the last.

1. The model does not write code; it echoes the file back. In 236 of 240 episodes the final `solution.py` was still the stub. The agent called `read.file`, then `write.file` with essentially what it had just read. That is what the 93–98% “wrote a file” rate was actually measuring. On the mutate tier, copy-with-one-token-changed *is* the correct fix, which is why it scored 59%. Given a stub there is nothing to tweak, so it copies verbatim. The four exceptions are incoherent: `defreduce(a, b):`, `defoperations`, and one sample in which the phrase “Terms of Service” bleeds in from pretraining.

2. Note the direction of the write-rate column. It is 93–98% on the *hard* tiers — *higher* than on the easy ones. The agentic workflow is genuinely learned: the model reads, edits, and verifies, with 0.07% malformed tool calls. It then writes wrong code with complete confidence. Tool use and coding are separable capabilities, and at 85M only the first is reachable.

3. RL on this tier is structurally impossible, not merely hard. Zero partial credit across 240 samples means every GRPO group has zero variance and is dropped: **0 update steps, by construction**. This is the identical failure mode the project already hit once (§3.1), arrived at from the opposite direction.

6.1 Why this took days to find

The tell was visible from the beginning and was nearly missed: the same checkpoint solves 9/300 (3%) of single-turn MBPP while reporting 70% on repository repair. A 20× gap between *writing* a solution and *repairing* one should be suspicious on sight. Two habits delayed it:

- **A rolling training loss cannot detect any of this.** It missed a real regression and twice signalled a plateau that was not one. Every quality claim in this report comes from a fixed held-out set instead — which is why Figure 1 plots the raw series underneath the smoothed one.
- **A benchmark’s difficulty is a parameter, and an unexamined parameter is an assumption.** The default breakage was a single regex substitution because that was the easiest thing to implement, and it silently became the definition of the task.

What was done about it, in code

`blackwell_lm/scenario.py` now takes a `difficulty` argument (`mutate/multi/stub/swap`), exposed as `--difficulty` on the RL script. A test asserts the property the easy tier lacked: a `stub` retains its signature but leaks no return and no variable from the original body, a `swap` never hands back the reference solution, and an `unknown` tier raises rather than silently falling back to the easy one — because a refactor that quietly reintroduced a hint would make the benchmark flattering again, and the number would look like progress.

7 The teacher-distillation plan, and why it was not built

The intended next stage was to have a strong agent generate trajectories for the 85M model to imitate. It was designed and then deliberately abandoned, on two independent grounds — either sufficient alone.

It would not have worked. The binding constraint is code-generation *capability*. Demonstrations teach the *shape* of `read` → `edit` → `verify`, which this policy already executes at 93–98%. They cannot install the ability to write correct Python into 85M parameters. Distillation would have produced a model imitating a strong agent’s edit structure while emitting wrong function bodies — precisely the current failure. And because `pass@12` is 0 with no partial credit, the RL stage that would follow the distillation cannot take a single update step regardless of the data’s quality. A 20-minute probe replaced an API budget and several GPU-hours.

It would also have been a terms violation. Using a Claude subscription to generate training data for a different model is a restricted use under Anthropic’s terms. This was flagged when the idea was first raised and did not change.

A related proposal — serving this checkpoint as a backend for a coding CLI — fails on a third, simpler ground: the checkpoint’s context is 2,048 tokens, and such a tool’s system prompt and tool schemas alone exceed that before the user’s first message. The model would be truncated out of its own instructions.

8 Reproducibility and artifacts

All artifacts are mirrored to `s3://nova-moe-checkpoints-045064753427` (54 objects, 22.8 GiB): 13 stage checkpoints, the tokenizer, per-run metrics as JSONL, the two audit probes and their logs, and a source tarball. Checkpoint integrity was verified byte-for-byte against the training host before that host was terminated.

The tokenizer is load-bearing

A *different* tokenizer silently invalidates every checkpoint — the weights load without error and produce nonsense. The exact tokenizer a run used is published alongside its weights, and its size was verified byte-for-byte (2,290,616 bytes) before teardown.

8.1 Operational notes worth carrying forward

- **Ghost spot requests hold quota.** An *active* one-time spot request pointing at an already-terminated instance consumed GPU quota and caused AWS to report `MaxSpotInstanceCountExceeded`. Roughly 15 launch attempts were misread as a capacity shortage; cancelling the ghost request launched an instance immediately. On teardown, cancel the request as well as the instance.
- **`NVTE_ALLOW_UNSAFE_PICKLE_EXTRA_STATE=1` must be set before importing Transformer Engine, or every *resume* crashes.** Cold starts look healthy, so this surfaces only after a spot reclaim — the worst possible moment.
- **Build TE with `NVTE_CUDA_ARCHS=120a`.** The released wheel silently disables FP4 stochastic rounding on `sm_120`. NVFP4 beat FP8 by only 8.6% end to end.

- **pgrep -f matches its own command line.** This produced a process watcher that reported a finished job as still running, indefinitely. Match on the interpreter and script (`pgrep -af "python3 /tmp/x.py"`) and confirm against GPU memory.
- **The sandbox is a robustness boundary, not a containment boundary.** It applies POSIX rlimits, an isolated interpreter, a throwaway directory, and process-group kills, but creates no network, mount, or PID namespace. Run the trainer in a container with no egress if that matters.

9 Conclusions

The pipeline is real and complete. Pretraining, SFT, GRPO, and multi-turn agentic RL all ran on real hardware, with FP32 master weights, MCP-shaped tools, verified scenarios across four difficulty tiers, a harness-owned grader, and durable metrics. That was the project’s stated goal: prove the pipeline, then point it at a real base model.

The model is at its ceiling, and the ceiling is narrower than it appeared. An 85M model at 240 tokens/param can convincingly learn an agentic *tool workflow* — 93–98% write rate, 86% self-verification, 0.07% malformed calls. It cannot learn to *write code*: 0/240 from a stub, 9/300 on MBPP. Those turned out to be separable capabilities, and no amount of reward shaping or teacher data changes the second one at this size.

Recommended next step. Run this same stack on an open-weights coding base in the 7B–30B range with a permissive licence, using `--difficulty stub` rather than `mutate` now that the easy tier is known to flatter. A model at that scale has non-zero pass@*k* on stub tasks, which means GRPO has variance to work with and can actually take update steps — the precondition this project’s final audit shows to be missing at 85M.

Of the results in this report, the one that changed the decision was the negative one.